Fatima Jinnah Women University

Department of Software Engineering

Cloud Computing (BSE-410) - Assignment 01 Solution

Git, Gitea, GitHub, Git LFS, and GitHub Pages

Student Name	Dua Malik
Registration No.	FA24B1-SE-034
Class / Section	BS(SE)-V B
GitHub Username	dua-malik612
Gitea Username	duamalik
Ubuntu Server IP	192.168.157.129
Submission Date	10 October 2026

Submission Links

- CC repository: https://github.com/dua-malik612/CC
- GitHub Assignment01: https://github.com/dua-malik612/Assignment01
- Gitea Assignment01: http://192.168.157.129:3000/duamalik/Assignment01
- Portfolio (GitHub Pages): https://dua-malik612.github.io/

Getting Started

Before starting the tasks, the screenshot directory was created inside the CC repository and the Ubuntu server prompt was verified to follow the dua-malik612@ubuntu:~$ pattern.

```
mkdir -p Assignments/Assignment01/screenshots
```

Git and Git LFS were confirmed installed on the Ubuntu server. Docker and Docker Compose were installed in Task 1.

Task 1: Install Gitea and Push a Repository from the Ubuntu Server

Step 1.1: Install and Run Gitea

Gitea is a lightweight self-hosted Git service. It was run with Docker Compose together with a PostgreSQL database container, using the instructor's compose.yaml without any changes.

Commands used:

```
sudo apt update -y
sudo apt install -y docker.io curl
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL
https://github.com/docker/compose/releases/latest/download/docker-compose-
linux-x86_64 \
  -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
sudo systemctl start docker
sudo usermod -aG docker $USER
exit    # sign out completely, then reconnect with ssh

git --version
docker --version
docker compose version

cd ~
git clone https://github.com/WaqasSaleem97/Gitea.git
cd Gitea
docker compose up -d
docker compose ps
curl -sS -o /dev/null -w "Gitea HTTP status: %{http_code}\n"
http://127.0.0.1:3000/
ip addr
```

Explanation:

- docker.io installs Docker Engine; the Compose plug-in was downloaded manually into the system-wide CLI plug-in directory so that the docker compose command works.
- usermod -aG docker adds the current user to the docker group so docker can run without sudo. A full sign-out and sign-in is needed for this to take effect.
- docker compose up -d starts the Gitea and PostgreSQL containers in the background; docker compose ps confirms both are running.
- The curl command checks that Gitea answers locally on port 3000 (expected HTTP status 200 or a redirect to the install page).

Gitea web installer settings:

Database type	PostgreSQL
Database host	db:5432
Database username	gitea
Database password	gitea
Database name	gitea
Server domain	192.168.157.129
Gitea base URL	http://192.168.157.129:3000/

After submitting the form, the administrator account was created and the installation completed. No password or token was shown in any screenshot.

Screenshot: task1_gitea_running.png



Step 1.2: Create and Push the Initial Repository

An empty public repository named Assignment01 was created in Gitea (no README, .gitignore, or license). A separate local repository was then created on the same server, outside the Gitea directory and the CC repository.

```
cd ~
mkdir Assignment01
cd Assignment01
git init
git branch -M main

echo "Name: Dua Malik" > README.md
echo "Registration Number: FA24B1-SE-034" >> README.md

git add README.md
git commit -m "Add student information"
```

A personal access token named ubuntu-server-token with write:repository permission was generated in Gitea (Settings > Applications > Manage Access Tokens). The token was copied once and was never placed in a command or remote URL.

```
git remote add gitea
http://duamalik@127.0.0.1:3000/duamalik/Assignment01.git
git remote -v
git push -u gitea main
# Password prompt: paste the personal access token (nothing is displayed
while pasting)
```

Explanation:

- git remote add only records the repository address; it does not contact Gitea or authenticate.
- The remote URL contains only the username. Embedding the token in the URL would save it in shell history and .git/config and would expose it in git remote -v.
- During git push, the token is entered at the password prompt and used as the password for the HTTP Git operation.

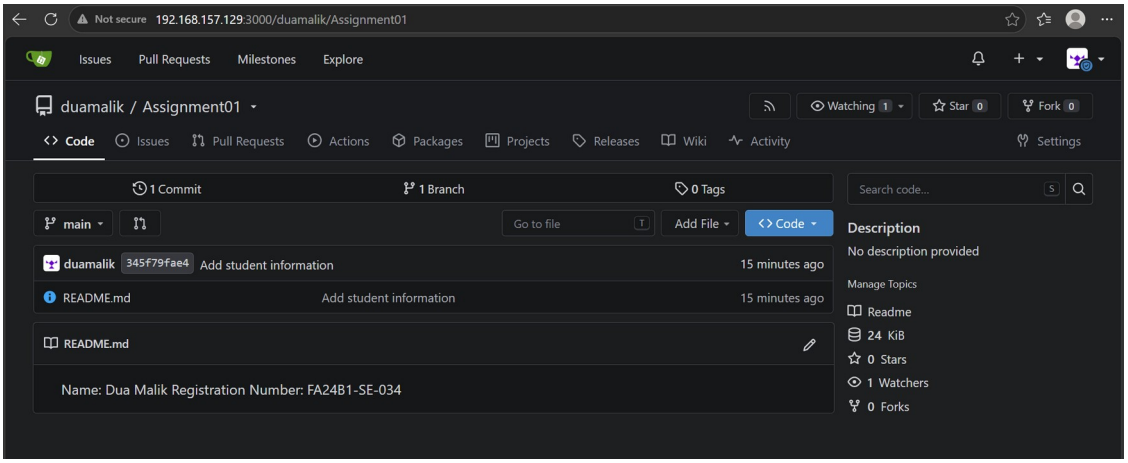
Screenshot: task1_gitea_push.png

```
dua-malik612@ubuntu:~/Assignment01$ git remote -v
gitea   http://duamalik@127.0.0.1:3000/duamalik/Assignment01.git
 (fetch)
gitea   http://duamalik@127.0.0.1:3000/duamalik/Assignment01.git
 (push)
dua-malik612@ubuntu:~/Assignment01$ git push -u gitea main
Password for 'http://duamalik@127.0.0.1:3000':
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 197 bytes | 39.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To http://127.0.0.1:3000/duamalik/Assignment01.git
   345f79f..403f1fa  main -> main
branch 'main' set up to track 'gitea/main'.
dua-malik612@ubuntu:~/Assignment01$
```

Step 1.3: Verify the Gitea Repository

The Assignment01 repository was opened in Gitea. The repository owner and name are visible, the repository is public, the rendered README.md shows my full name and registration number, and the Gitea URL is visible in the browser.

Screenshot: task1_gitea_repository.png



Task 1 result: Gitea was installed and run on the Ubuntu server, a separate local repository was created there, and it was pushed to Gitea over the server's loopback interface (127.0.0.1:3000).

Task 2: Push the Same Repository to GitHub

Step 2.1: Create the GitHub Repository and Verify Both Remotes

An empty public repository named Assignment01 was created on GitHub (no README, .gitignore, or license). GitHub was then added as a second remote of the same local repository.

```
cd ~/Assignment01
git remote add github https://github.com/dua-malik612/Assignment01.git
git remote -v
```

The output lists both gitea and github with their fetch and push URLs. Neither URL contains a token or password.

Screenshot: task2_remotes.png

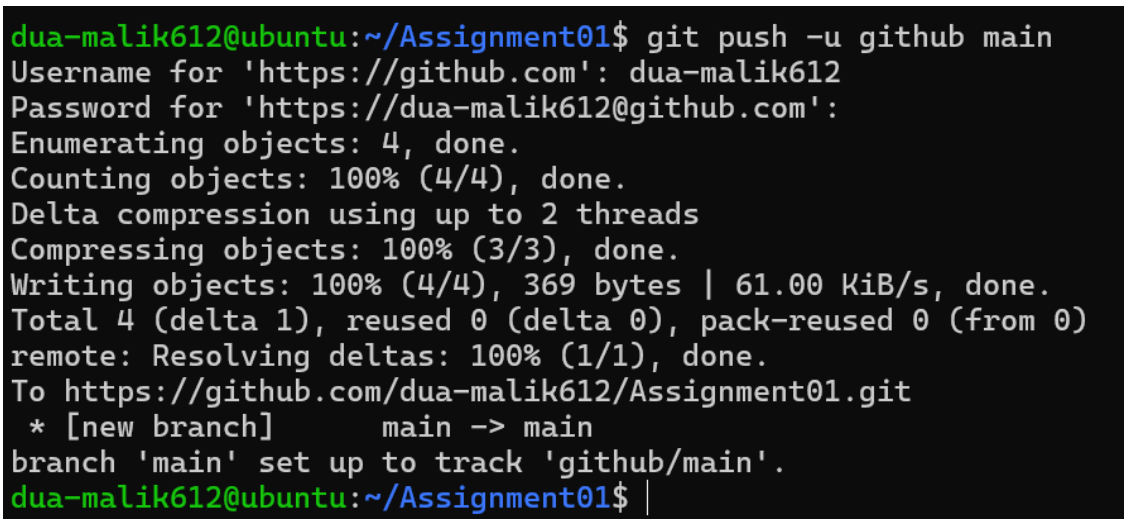


Step 2.2: Push to GitHub

```
git push -u github main
```

Git remotes are independent named addresses, so the same local main branch can be pushed to multiple servers. GitHub authentication was done using a personal access token / credential helper without exposing the credential on screen.

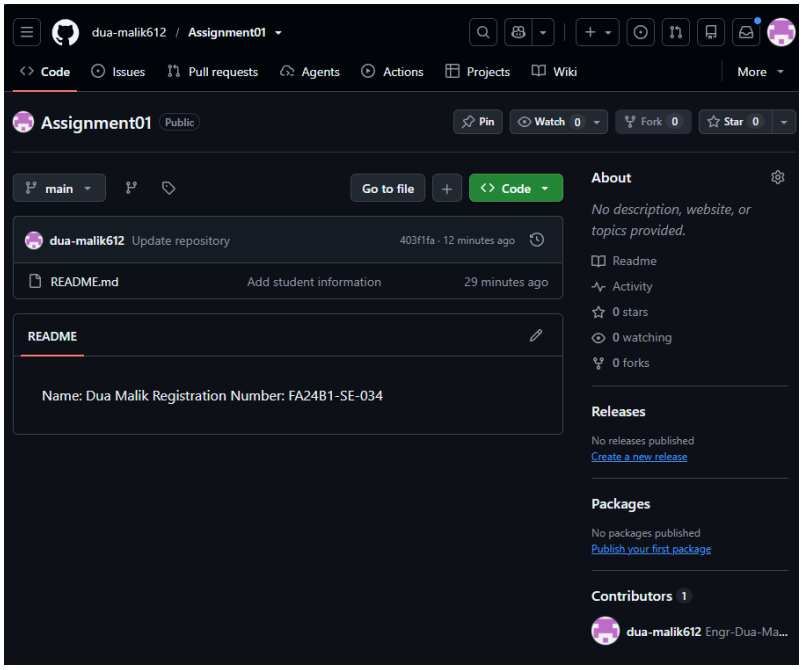
Screenshot: task2_github_push.png



Step 2.3: Verify the GitHub Repository

https://github.com/dua-malik612/Assignment01 was opened. The owner is my GitHub account, the repository name is Assignment01, the repository is public, and the rendered README shows my full name and registration number.

Screenshot: task2_github_repository.png



Task 2 result: The same local repository is connected to both Gitea and GitHub and is publicly available on GitHub.

Task 3: Track Three Large Files with Git LFS

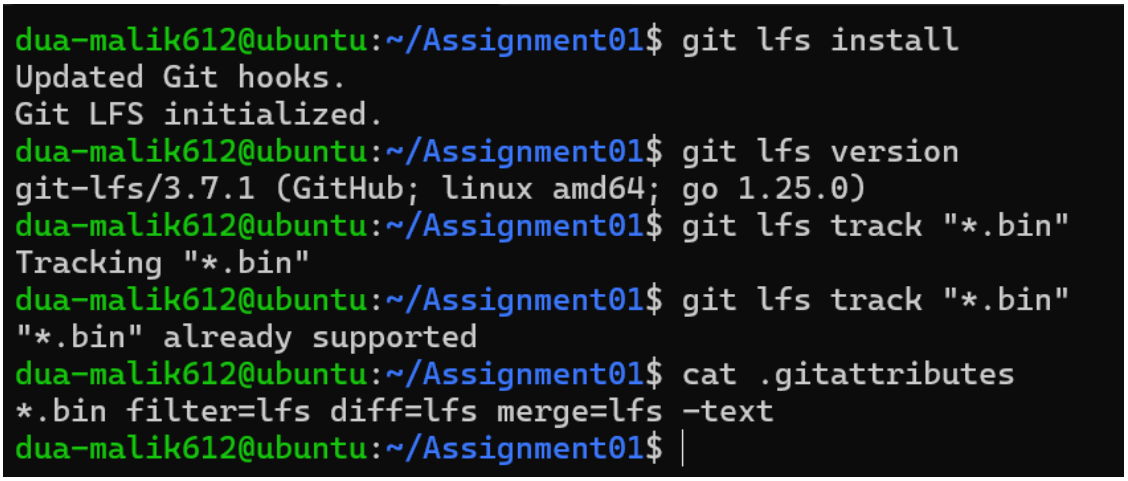
Step 3.1: Configure Git LFS

Git LFS (Large File Storage) replaces large files in the Git history with small pointer files and stores the real content on a separate LFS server. This is needed because GitHub rejects normal files larger than 100 MiB.

```
cd ~/Assignment01
git lfs install
git lfs version
git lfs track "*.bin"
cat .gitattributes
```

git lfs install sets up the LFS hooks, and git lfs track "*.bin" writes a rule to .gitattributes so every .bin file is handled by LFS. Expected rule: *.bin filter=lfs diff=lfs merge=lfs -text

Screenshot: task3_lfs_setup.png

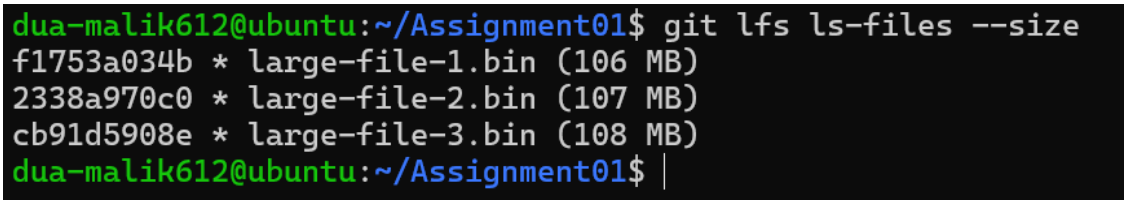


Step 3.2: Create and Verify Three LFS Files

```
truncate -s 101M large-file-1.bin
truncate -s 102M large-file-2.bin
truncate -s 103M large-file-3.bin
git add .gitattributes large-file-1.bin large-file-2.bin large-file-3.bin
git lfs ls-files --size
```

truncate creates sparse files of the requested sizes (101, 102 and 103 MiB), each above the 100 MiB limit. git lfs ls-files --size lists all three files as LFS-tracked together with their sizes.

Screenshot: task3_lfs_files.png

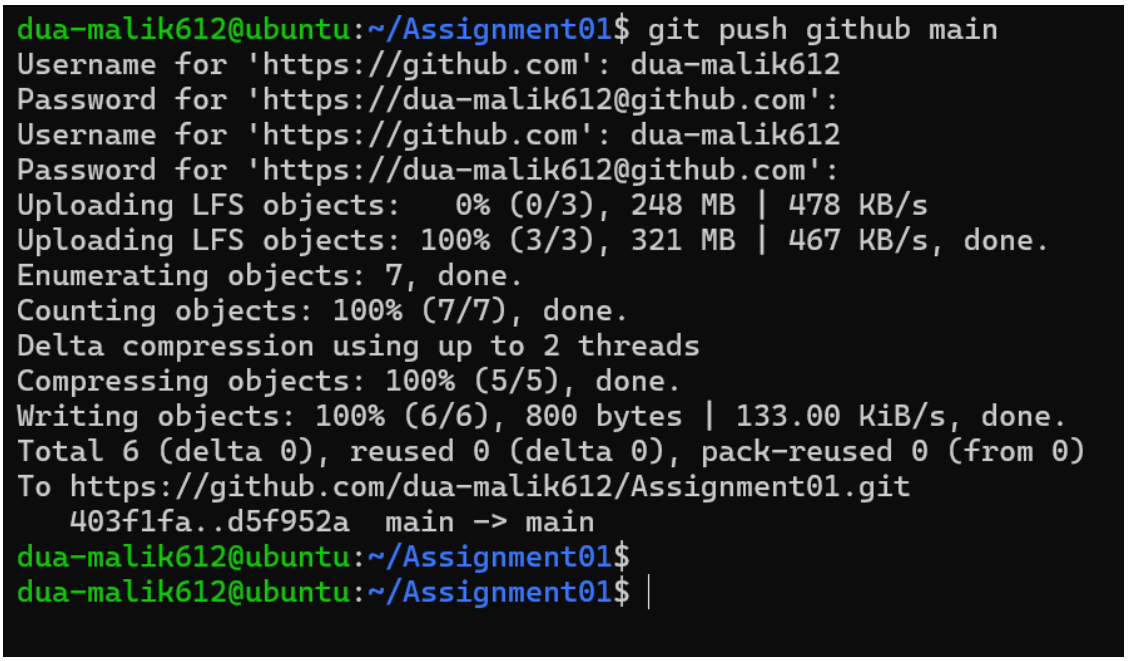


Step 3.3: Commit and Push the LFS Files

```
git commit -m "Add three files using Git LFS"
git push github main
```

The LFS objects are uploaded first and then the commit containing the pointer files is pushed to GitHub. The first successful upload was captured, showing the LFS upload completing (3/3).

Screenshot: task3_lfs_push.png



Task 3 result: Three files larger than 100 MiB are tracked with Git LFS and pushed to the GitHub Assignment01 repository.

Task 4: Create a Portfolio or CV with GitHub Pages

Step 4.1: Create and Push the Portfolio Repository

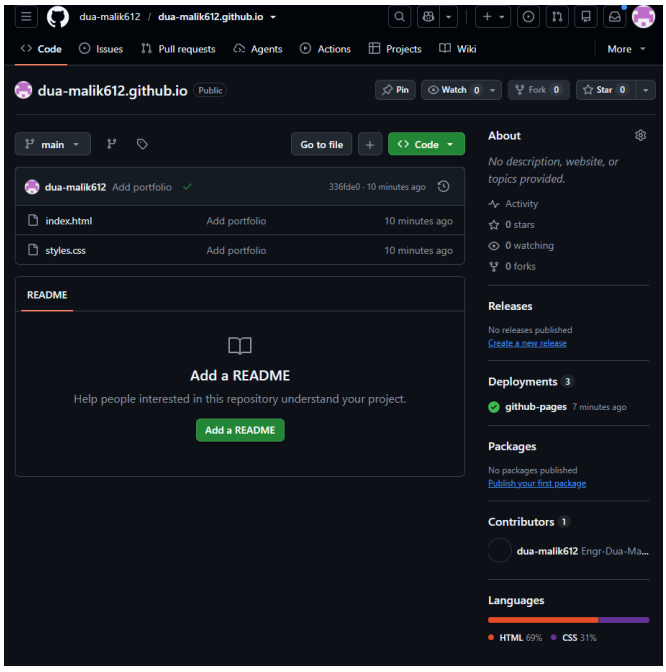
A new public GitHub repository named dua-malik612.github.io was created. The portfolio was built using HTML and CSS with the following files:

- index.html - the portfolio page containing the About Me, Education, Skills, and Projects sections.
- styles.css - the styling for the page.

The source files were committed and pushed to the main branch.

```
git init
git branch -M main
git add index.html styles.css
git commit -m "Add portfolio"
git remote add origin https://github.com/dua-malik612/dua-
malik612.github.io.git
git push -u origin main
# remote already had an auto-created index.html, so the push was rejected;
# the portfolio version was pushed with:
git push -u origin main --force
```

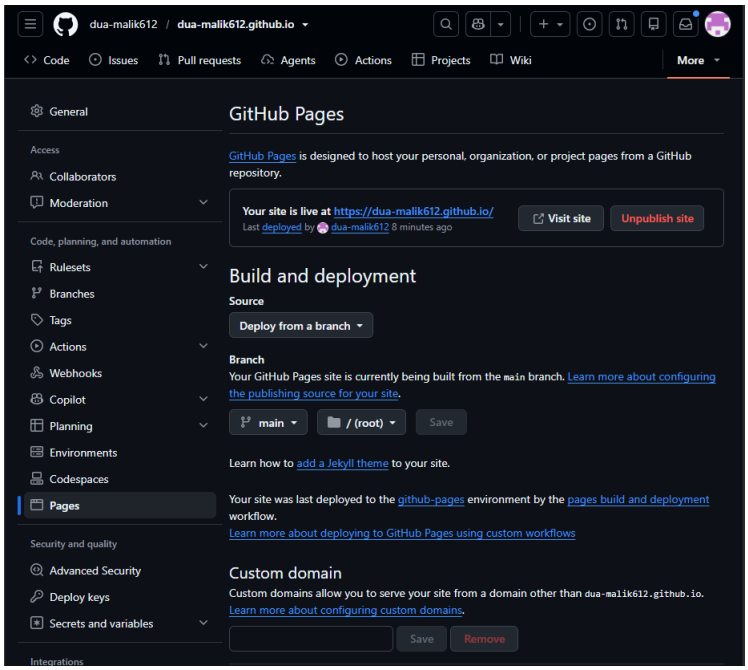
Screenshot: task4_pages_repository.png



Step 4.2: Publish with GitHub Pages

In the portfolio repository, Settings > Pages was opened. Under Build and deployment, Source was set to Deploy from a branch, the main branch and / (root) folder were selected, and the settings were saved. The page was refreshed until GitHub reported that the site is live.

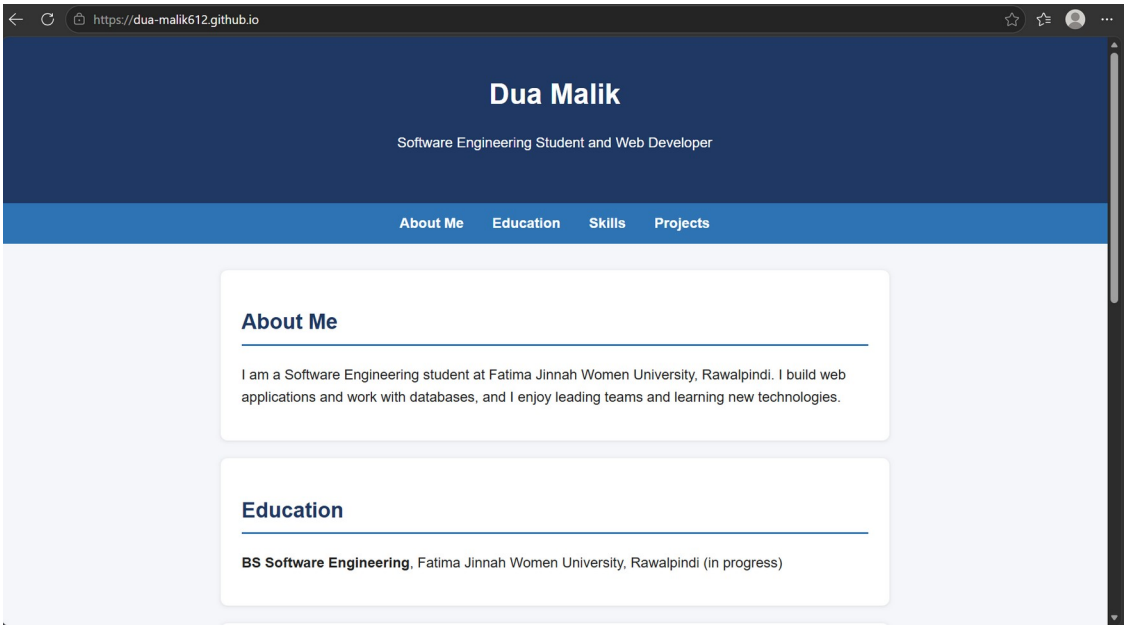
Screenshot: task4_pages_deployment.png



Step 4.3: Verify the Live Portfolio

https://dua-malik612.github.io/ was opened in the browser and loaded successfully, showing the portfolio sections.

Screenshot: task4_portfolio_live.png



Task 4 result: The portfolio is stored in a public GitHub repository and published as a live GitHub Pages site.

Assignment Summary

- Installed and ran a Gitea service on an Ubuntu server using Docker Compose.
- Performed all Git work from the Ubuntu server with my GitHub identity visible in the prompt.
- Pushed the same local repository to Gitea and GitHub using two remotes, authenticating without exposing credentials.
- Configured Git LFS and pushed three files larger than 100 MiB to GitHub.
- Created and published a portfolio with GitHub Pages.
- Prepared this Word solution and exported it as a PDF.

Final Submission Checklist

- Assignment01.md, Assignment01_Solution.docx and Assignment01_Solution.pdf are in CC/Assignments/Assignment01/.
- All 12 screenshots are in CC/Assignments/Assignment01/screenshots/ and are readable.
- No screenshot contains a token, password, or other credential.
- CC, GitHub Assignment01, and dua-malik612.github.io repositories are public; all use the main branch.
- Gitea Assignment01 was created and pushed from the Ubuntu server.
- Files and links submitted on Google Classroom before the deadline.